Secure Vibe Delivery

Simple checks every AI-built project passes before it goes live for a client

ESSENTIAL

Security

Is it safe? No leaked keys, no risky code, no unsafe libraries, users see only their own data.

REQUIRED

Functional

Does it work? Features behave as expected, including login and permissions.

REQUIRED

Performance

Can it cope? The app stays fast and stable with the expected number of users.

Code that works is not the same as code that is safe

What recent studies found about AI-written code

1 in 10

AI-agent solutions that worked were also secure

CMU SusVibes study, 2025

45%

of AI-written code samples failed security tests

Veracode, 2025

Same flaws

ChatGPT, Copilot and Gemini made the same mistakes from the same request

Kahn & Hastings, 2026

1 in 5

libraries suggested by AI did not exist; attackers register these names

USENIX Security, 2025

What this means for us: the AI tool doesn't matter. Every AI-built delivery gets the same checks, and a person owns the result.

How researchers test AI-written code, and how we will

Every change is checked three ways before it counts as done

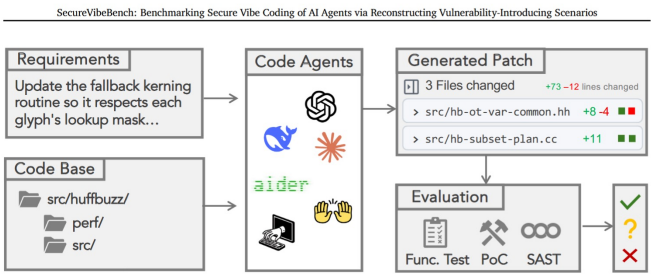


Figure 1: Illustration of SECUREVIBEBENCH.

Source: SecureVibeBench (arXiv 2509.22097). Best agent: only 23.8% of answers were both correct and secure.

Does it work?

Functional tests: pytest or Jest

Can it be broken?

Try to misuse it: wrong user, bad input

Does a scanner flag it?

Semgrep, Snyk, CodeQL

We add one more: Can it handle real traffic? Load test with Locust.

Our flow: from AI agent to go-live

Each step must pass before the next one starts

AI coding agent

Writes or changes the code

Semgrep

Scans our code, plus our own rules for common AI mistakes

Snyk

Code, libraries, secrets, cloud setup and containers

Tests and review

pytest / Jest, Locust load test, a person reviews

Go-live gate

Go only when all must-pass checks pass, or an exception is approved

Any failure goes back to fix, then everything is scanned again

Rule: a scan that did not run counts as a failure, never as a pass.

Three kinds of checks, clearly split

Security is essential: an open security failure always stops go-live

Security — ESSENTIAL

Is it safe?

Semgrep | Snyk | CodeQL*

- No passwords or keys left in the code
- No risky code patterns
- No libraries with known security holes
- No fake or unknown libraries
- Users can only see their own data

Functional — REQUIRED

Does it work?

pytest (Python) | Jest (JavaScript)

- Each feature does what was asked
- Parts work together correctly
- Main user journeys work end to end
- Login and permissions behave correctly

Performance — REQUIRED

Can it cope?

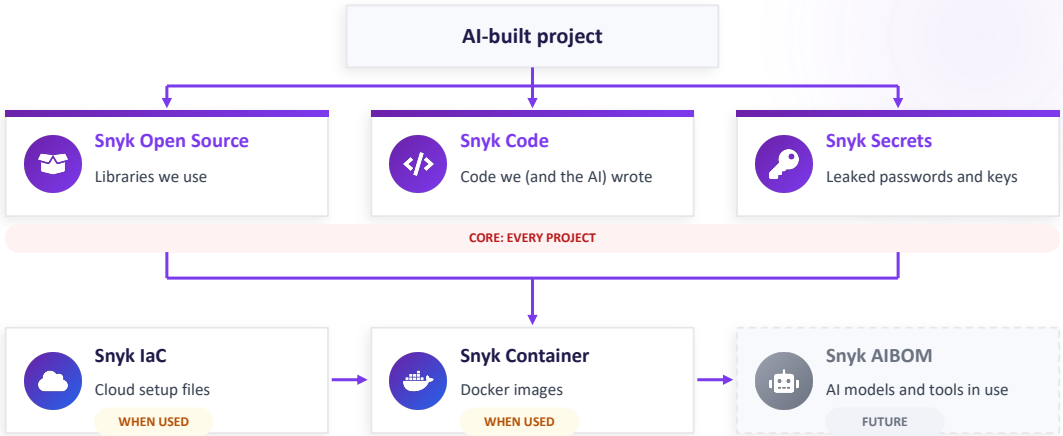
Locust

- Handles the expected number of users
- Pages and APIs respond fast enough
- No errors or crashes under load
- AI features stay within usage limits

** CodeQL when needed, mainly for client production and sensitive data*

Security checks: the Snyk flow

Run what applies to the project. Core scans run on every project



Semgrep and CodeQL: a second pair of eyes

Two scanners catch more than one. No single tool proves the code is safe

Semgrep

CORE

Fast scan of our code on every change

- Standard security rules (OWASP Top 10)
- Our own rules for mistakes AI tools often make
- Runs on the laptop and on every pull request

Snyk Code

CORE

Code scan inside Snyk (enabled for us)

- Finds risky code and shows how to fix it
- Same dashboard as library and secret results
- Can keep watching after go-live

CodeQL

WHEN NEEDED

Deeper check that follows data through the code

- Use for client production and sensitive data
- Free on GitHub for public repos
- Slower, so run on pull requests, not the laptop

Examples our own rules will catch: running text as code, switching off HTTPS checks, building database queries by joining strings.

Security: what must pass

Essential. If any of these fail, the project does not go live

No passwords or keys in the code	0 found	Snyk Secrets
No serious problems in our code	0 critical or high	Semgrep, Snyk Code
No serious problems in libraries	0 critical; fix highs that have a fix	Snyk Open Source
Every library is real and well known	No unknown or brand-new packages	Snyk + review
Users only see their own data	Tested on every page and API	pytest / Jest tests
A person reviewed the change	Not the person who prompted the AI	Pull request

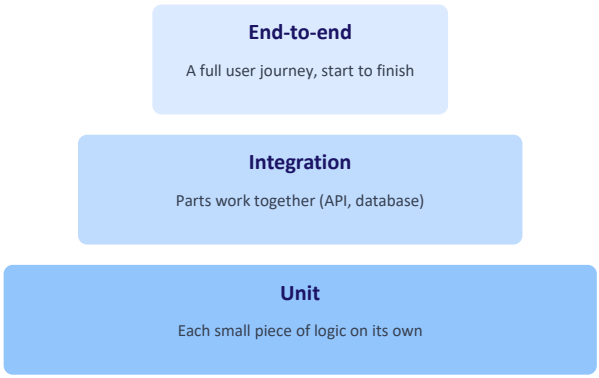
Exceptions

- Only for a real reason, written down
- A named owner and an end date (90 days max)
- Approved by the Delivery Lead
- Never just to get a green build

Never: skip a scan to save time.

Functional testing: does it work?

pytest for Python, Jest for JavaScript. AI can write the tests; a person checks them



Fewer, slower tests at the top; many fast tests at the base

Rules for every project

- All tests pass before merging
- Tests cover at least 70% of new code
- Always test the wrong cases: not logged in, another user's data, a lower role
- AI-written tests are reviewed: they must test what was asked, not just what the code does
- Tests run automatically on every pull request

Performance testing with Locust

Simulate real users before real users arrive

Pick journeys
The 3-5 things users do most

Add users
Ramp up to 1.5x the expected peak

Hold it
Run steadily for 15 minutes

Check results
Speed, errors, stability

< 1%
requests fail

Within target
response time agreed with the client

0
crashes or restarts

For AI features: also check usage limits and cost. A busy app should slow down politely, not fail or run up a large bill.

When each check runs

Fast checks early and often; the full set before go-live

On the laptop
Before code is saved

SECURITY
- Semgrep (quick)
- Secret check

FUNCTIONAL
- pytest / Jest (changed parts)

Pull request
Every change, automatic

SECURITY
- Semgrep
- Snyk Code, Open Source, Secrets
- CodeQL (when needed)

FUNCTIONAL
- All pytest / Jest tests

Before go-live
On the test environment

SECURITY
- Full Snyk scan
- Data-access tests

FUNCTIONAL
- End-to-end tests

PERFORMANCE
- Locust load test

After go-live
Ongoing

SECURITY
- Snyk monitor: alerts on new library issues

PERFORMANCE
- Watch speed and errors

Go-live checklist

One page. Signed by the Delivery Lead before release

Security (must pass)
- [ ] No keys or passwords in code
- [ ] 0 critical / high in our code
- [ ] 0 critical in libraries
- [ ] All libraries real and known
- [ ] Users see only their own data
- [ ] Change reviewed by a second person

Functional
- [ ] All pytest / Jest tests pass
- [ ] 70%+ of new code tested
- [ ] Main user journeys pass end to end
- [ ] Login and permission tests pass

Performance
- [ ] Locust run at 1.5x expected users
- [ ] Under 1% failed requests
- [ ] Response time within target
- [ ] No crashes under load

Sign-off: Delivery Lead confirms all boxes, attaches the scan and test reports, and records any approved exceptions.

Roadmap: start with the core, extend later

The initial roadmap covers what every project needs; the rest follows once the basics are routine

Initial roadmap (weeks 1-6): core checks

SECURITY

- Snyk: Open Source, Code, Secrets
- Semgrep with our AI-mistake rules
- CodeQL for client production when needed

FUNCTIONAL

- pytest / Jest on every pull request
- Data-access and login tests

PERFORMANCE

- Locust load test before go-live

Plus: AI rules file for coding agents, go-live checklist, team training.

Future roadmap: extended checks

- Snyk IaC and Container (where used)
- Snyk AIBOM: list of AI models and tools
- Live attack test on the website (OWASP ZAP)
- Security reviewer agent and Claude skill
- VS Code plugin for in-editor checks
- Approved list of AI tools and plug-ins
- Longer load tests for regulated clients

Next steps

What we need to start

1	Agree the approach	Security checks are essential for every client delivery
2	Set up the pipeline	Snyk, Semgrep and tests running on all active projects (warning mode first)
3	Train the team	2-hour session: AI Testing, Security and Secure Vibe Coding
4	First go-live	One client project passes the full checklist

Appendix: commands for the team

Run locally or in the pipeline. Check exact options against current Snyk and Semgrep docs

Check	Command
Security	`snyk test --all-projects` (libraries)
	`snyk code test` (our code)
	`snyk secrets test` (keys and passwords, where on our plan)
	`semgrep scan --config auto --config ./svd-rules`
	`snyk monitor` (after go-live)
Functional	`pytest --cov | npx jest --coverage`
Performance	`locust -f locustfile.py --headless -u <users> -r <rate> -t 15m --host <test URL>`

Sources

- SecureVibeBench, arXiv 2509.22097 (figure on slide 3)
- SusVibes (CMU), arXiv 2512.03262
- Veracode GenAI Code Security Report, 2025
- Kahn & Hastings, arXiv 2607.20713, 2026
- Spracklen et al., package hallucination, USENIX Security 2025
- Project research: Gemini report on AI code security; Semgrep and Snyk handover
- Palo Alto Networks, "Let AI Coding Do More with You in Control", 2026
- Microsoft Learn: load testing a chat app with Locust
- Betta Lyon Delsordo, insecure-vibes; Autonoma and LevelUp articles on vibe-coding testing